

PROJECT TITLE

BMO — An Autonomous Interactive Mini Robot for Enhanced Human-Robot Interaction

Prepared by:

IOT TEAM	AI TEAM	FLUTTER TEAM
-Fatma Khaled -Fatma Saed -Kenzi Ahmed -Morooj Mohamed -Toka Elghazy -Takwa Shabaan -Habiba Alsayed	-Haneen Tarek -Mahmoud Hossam -Arsanious Nabil -Rana Ahmed	-Ashrakat Ahmed -Fam Emad

Supervised by: ROBOTECH

Submission Date: 20 – 4 – 2026

Abstract

This document presents the design, development, and implementation of BMO — a smart, interactive mini-robot engineered to facilitate simple, engaging, and meaningful human-robot interaction and bridge the gap between human users and autonomous robotic systems. BMO integrates hardware, software, and artificial intelligence into a unified platform that enables real-time face recognition, object detection and classification, gesture-based interaction, and app-controlled movement, all managed through a dedicated Flutter mobile application.

The project was executed collaboratively by three specialized teams — Artificial Intelligence (AI), Internet of Things (IoT), and Mobile Application (Flutter) — working in three structured phases from initial research through full system integration.

BMO demonstrates that a compact robotic platform can deliver meaningful and adaptive human-robot interaction when hardware constraints are addressed through thoughtful software architecture. The system is designed for extensibility.

1. Introduction

The field of human-robot interaction (HRI) has grown substantially in recent years, driven by advances in computer vision and embedded systems. Yet the majority of capable robotic platforms remain large, expensive, and inaccessible outside of research or industrial settings. BMO was conceived to challenge this norm — a mini-robot that is compact and affordable, yet capable of delivering rich, responsive interaction with the humans around it.

BMO can recognize the faces of people she has previously met and greet them by name, detect and announce the objects in her environment, respond to a hand raised toward her by initiating a handshake gesture, and be controlled remotely through a built mobile application. Her self-balancing mechanism ensures stable operation on flat surfaces, and she is powered by a rechargeable battery for untethered deployment.

This report documents the full technical journey of BMO's development: the problem that motivated the project, the system architecture that was designed to address it, the methodologies employed by each team, the implementation details of every subsystem, and the results achieved across three development phases.

2. Problem Statement

In today's rapidly evolving technological landscape, there is an increasing demand for interactive and intelligent robotic systems that can effectively engage with users while performing multiple functions. However, many existing educational and personal robots are limited in their capabilities, offering only basic movements or pre-programmed actions without meaningful interaction, adaptability, or user control. This lack of functionality reduces user engagement and limits the potential of such systems in both learning environments and everyday applications.

Additionally, most robots do not integrate multiple input and output features in a seamless and user-friendly manner. They often lack responsiveness to real-time human interaction, such as touch or environmental changes, and do not provide feedback that enhances the user experience. Furthermore, the absence of remote-control options and intelligent object recognition features restricts their usability and innovation potential.

Therefore, there is a need to develop a multifunctional mini robot that combines interactive sensors, display systems, and intelligent features into one cohesive design. The robot should be capable of responding to touch inputs, performing various movements, displaying expressions, and identifying objects using artificial intelligence. It should also provide flexible control options, including both manual and automatic modes through a dedicated application. By addressing these limitations, the project aims to create a more engaging, responsive, and versatile robotic system.

that enhances user interaction and demonstrates the practical integration of modern technologies.

3. System Overview

BMO's system integrates three core technological domains: Artificial Intelligence (AI), Internet of Things (IoT), and mobile application development. The system operates on a client-server architecture where the robot and the user's mobile device serve as clients communicating with a centralized FastAPI server that hosts the robot's intelligence.

3.1 Operating Modes

- **Object Detection and Classification Mode:** The robot continuously identifies objects in its field of view using a fine-tuned YOLO model and announces their names via text-to-speech.
- **Face Identification and Handshake:** The robot recognizes known individuals and greets them by name; raises a handshake gesture when an open palm is presented toward the camera.
- **Guest Registration:** The Flutter app transmits a photograph and name; the AI pipeline extracts a facial embedding and stores it in the face database for future recognition.

3.2 Communication Architecture

Channel	Protocol	Description
IoT ↔ AI Server	WebSocket	The ESP32-CAM streams JPEG frames continuously to the FastAPI server over a persistent WebSocket connection (/ws/esp_cam). The audio ESP32 maintains a separate WebSocket (/ws/esp_audio) to receive synthesized PCM audio on demand.
Flutter ↔ AI Server	HTTP POST	The Flutter app sends mode-change and face-registration commands to the /flutter endpoint via HTTP POST, with a mode header (F, O, or R) and optional JSON body.
Flutter ↔ ESP32	WebSocket	Real-time directional movement commands (F/B/L/R/S) are sent directly from the Flutter app to the IoT ESP32 over a persistent WebSocket connection for minimal latency.

4. Goals and Objectives

- Develop a compact, self-balancing robotic platform capable of stable operation on flat surfaces without external support.
- Implement a real-time face recognition pipeline that correctly identifies registered individuals with high accuracy.
- Integrate a YOLO-based object detection model fine-tuned on a custom dataset relevant to the robot's deployment environment.
- Enable gesture-based interaction through MediaPipe hand detection, allowing the robot to respond to an open-palm handshake gesture.
- Deliver a seamless Flutter mobile application that provides mode selection, face registration, movement control, and status feedback.
- Establish a robust HTTP/WebSocket communication protocol enabling real-time coordination between the mobile app, AI server, and hardware.
- Design the AI pipeline to be parallelized — face recognition and hand detection must run concurrently without blocking the video stream.
- Implement on-device text-to-speech synthesis so the robot can communicate results audibly through its speaker.
- Build an extensible face database supporting live registration and hot-reloading without server restart.

5. Methodology

The project followed an iterative, milestone-driven development methodology divided into three phases. Each phase had defined deliverables for each of the three teams.

5.1 AI Development Methodology

The AI team adopted a pipeline-based development approach, building each module independently and then integrating them under a unified server. Unit-level testing was used to validate each component before integration. The face recognition pipeline was designed around the widely-adopted FaceNet architecture, using the facenet-pytorch library's InceptionResnetV1 model pre-trained on the VGGFace2 dataset, which provides strong generalization to new identities. Object detection was addressed with Ultralytics YOLOv8 fine-tuned on a custom-collected dataset.

A key methodological decision was to parallelize the two most compute-intensive tasks — face recognition and hand detection — using Python's multiprocessing module with dedicated worker processes and non-blocking queue-based communication. This ensures that neither task starves the other of CPU time and that the video streaming loop is never blocked waiting for inference results.

5.2 IoT Development Methodology

The IoT team was responsible for the hardware layer, covering motor control, self-balancing, sensor integration, and embedded communication. Development followed an iterative hardware-first approach: individual components (motors, IMU, ESP32-CAM, audio ESP32) were validated independently before integration into the full chassis. The self-balancing PID controller was tuned empirically to achieve stable upright operation. WebSocket firmware was implemented on the ESP32-CAM to maintain a persistent connection with the AI server.

5.3 Flutter Development Methodology

The Flutter team evaluated state management options and selected `flutter_bloc` Cubits for their simplicity, testability, and clear separation of UI from business logic. A feature-based folder structure was adopted, with a shared widget library enforcing visual consistency across all screens.

The team defined a dual-channel communication strategy in coordination with the IoT and AI teams: HTTP POST for mode activation and face registration (low frequency), and WebSocket for real-time directional movement commands (high frequency, button-press granularity). Network addresses were centralized in `ApiConstants` so that changing the server IP requires editing a single file.

The visual language — glassmorphism cards, pill-shaped buttons, Inter font at `FontWeight.w900`, a blue gradient palette, and orange icon accents — was designed in Phase 1 and implemented consistently across all screens. Each screen delegates business logic to its corresponding Cubit, keeping widget trees purely declarative.

6. System Architecture

The BMO system is structured around three loosely-coupled layers that communicate over a shared local network. This section describes the architecture of each layer and the interfaces between them.

6.1 Hardware Layer (IoT)

The hardware layer consists of the physical robot chassis and its embedded electronics. The self-balancing platform integrates an IMU for orientation feedback, a PID controller running on the main microcontroller for motor torque adjustment, and two drive motors for locomotion. The ESP32-CAM module captures a continuous JPEG video stream and streams it to the AI server over WebSocket. A separate audio ESP32 receives PCM audio data from the AI server and drives the robot's speaker. The IoT layer receives directional movement commands from the Flutter app via WebSocket and from the AI server's mode transitions.

6.2 AI Server Layer

The AI server is the computational core of BMO. It is implemented as an async FastAPI application that exposes three endpoints:

- **WebSocket /ws/esp_cam:** Receives a continuous stream of JPEG frames from the ESP32-CAM. Dispatches each frame to the appropriate processing pipeline based on the current mode. Sends synthesized PCM audio to the audio WebSocket when a result is produced.
- **WebSocket /ws/esp_audio:** Maintains a persistent connection with the audio ESP32. Receives any keep-alive signals and holds the connection open to push audio data on demand.
- **HTTP POST /flutter:** Accepts HTTP POST requests from the Flutter application. Reads the mode header (F, O, or R) and runs the desired mode.

6.3 Mobile Application Layer (Flutter)

The Flutter mobile application is the primary human interface for BMO. It communicates with both the AI server (via HTTP POST) and the IoT ESP32 hardware (via WebSocket) in parallel, enabling both high-level mode control and low-latency real-time movement.

6.3.1 Design System

Color Palette (app_colors.dart): All colors are centralized in AppColors — no hex literals appear in screen code.

Token	Hex	Usage
primaryColor	#86B4E5	Buttons, nav bar, gradients, title text
secondaryColor	#FFFFFF	Background end-stop, card fill
linearColor	#7EB7BE	Accent / linear gradient secondary stop
textColor	#434343	Primary body text, dark labels
textColor2	#E5E7E9	Light text on colored buttons
icons	#F29D38	All icon tints (orange accent)

Typography: Font family: Inter (custom asset), weight FontWeight.w900 throughout. Title: 54 px, primaryColor, drop-shadow (blur 10, offset y+4). Body/buttons: 22–32 px depending on context. Error text: 18 px, Colors.red.

Visual Language: Pill-shaped buttons (borderRadius: 50) for all primary actions. Glassmorphism cards using BackdropFilter blur (sigmaX:20, sigmaY:10) + semi-transparent fill + white border. Gradient backgrounds: RadialGradient on the Start screen (oval-transformed), LinearGradient on the Home screen. Consistent 1.5 px white border on interactive containers.

6.3.2 Architecture Pattern

The application follows a feature-based folder structure combined with the BLoC (Business Logic Component) state management pattern using flutter_bloc Cubits.

Layer	Responsibility
Presentation	Screens & widgets — renders UI, dispatches Cubit events
Logic (Cubit)	Business logic — calls data layer, emits typed states
Data (Repo)	Wraps ApiService / SocketService — performs network I/O
Core	App-wide constants (ApiConstants), colors, HTTP & WS clients
Shared	Reusable widget library used across all features

6.3.3 Navigation Flow

Start → BottomNavigationCustom (4 tabs). Modal screens are pushed via MaterialPageRoute.

Tab	Label	Icon	Destination
0	Robot	smart_toy	HomeScreen
1	Moving	control_camera	MovementScreen
2	Info	my_library_books_rounded	InformationScreen
3	Camera	camera	CameraScreen → (FaceIDScreen CameraModesScreen)

7. Project Phases

Phase 1 — Research, Planning & Component Selection

The first phase focused on establishing the foundations of the project across all three teams.

AI Team

- Surveyed face recognition architectures, settling on FaceNet (InceptionResnetV1 / VGGFace2) as the embedding backbone based on its balance of accuracy and inference speed on CPU.
- Evaluated face detection options — dlib, MTCNN, and MediaPipe Face Mesh — selecting MediaPipe for its lightweight real-time performance and the quality of its 468-point landmark mesh for alignment.
- Identified YOLO (Ultralytics YOLOv8) as the object detection framework and began collecting and annotating a custom dataset tailored to the robot's expected environment.

- Assessed text-to-speech options, comparing pyttsx3 (fast, offline) with Coqui XTTS (higher quality, neural) to determine the best fit for latency-constrained real-time output.

IoT Team

- Researched the project concept and defined the core features of "BMO", the Mini Robot, including self-introduction, a student committee system where the robot provides audio information about each committee (IoT, ROV, AI, Flutter, and Media) via the app, movement control, self-balancing, touch response, object detection, face identification, and situation-based facial expressions via the OLED display.
- IoT, AI, and Flutter integration points were identified and aligned across the team.

Flutter Team

- Evaluated state management options; selected flutter_bloc Cubits for simplicity and testability.
- Agreed on feature-based folder structure and shared widget library contract with the team.
- Defined the dual-channel communication strategy (HTTP + WebSocket) in coordination with IoT and AI teams.
- Designed the visual language: glassmorphism cards, pill buttons, Inter font, blue gradient palette, and orange icon accent.
- Defined ApiConstants structure and confirmed IP/port assignments with the IoT team.
- Selected packages: flutter_bloc, http, web_socket_channel, image_picker, google_nav_bar, gap, vector_math.

Phase 2 — Core Implementation

Phase 2 was the primary implementation phase. Each team built their core deliverables in parallel, with agreed interfaces allowing independent progress.

AI Team

- Implemented the complete face recognition pipeline: MediaPipe-based face detection and landmark extraction, eye-center-based rotation alignment, padded bounding box crop, resize to 160×160, pixel normalization to $[-1, 1]$, and InceptionResnetV1 embedding extraction.
- Built the JSON-backed face database with save, search (with L2 comparison at threshold 0.8), and hot-reload functions.

- Trained and integrated the YOLOv8 object detection model on the collected dataset, achieving reliable detection.
- Implemented the MediaPipe Hands-based handshake detector, classifying high-five gestures by counting extended fingers, measuring finger spread, and validating hand orientation.
- Developed the text-to-speech module with support for pyttsx3 and a pathway for XTTS, outputting raw PCM bytes for WebSocket streaming.
- Implemented the Guest Registration coordinator and tested it end-to-end.

IoT Team

- Building on the features defined in Phase 1, hardware components were finalized and technical resources were analyzed.
- The code architecture was designed with clearly defined functions per module, with tasks distributed among team members.
- Movement control options via the Flutter app were defined, enabling manual and selection-based navigation modes.
- TTS integration was also explored in coordination with the AI team to support object detection and face identification features.

Flutter Team

- Implemented the complete shared widget library: CustomButton, CustomGlassBox, CustomTextField, CustomNavigationBottom, CustomText, CustomTitle, CustomModesButtons, OvalGradientTransform.
- Built ApiConstants, ApiService (HTTP POST + GET), and SocketService (WebSocket connect / send / disconnect).
- Implemented AppColors color token system.
- Built Start, Home, Movement, Information, and Feedback screens.
- Implemented MovementCubit (startDanceParty → dance command, stopMovement → stop).
- Built SwitchCustom with WebSocket-based real-time D-pad directional control; integrated onTapDown / onTapUp auto-stop.
- Implemented CameraCubit with activateMode(), activateGreeting(), registerUser(), and resetState().
- Built the full Camera feature flow: Camera entry screen → CameraModesScreen → FaceIDScreen.
- Integrated ImagePicker for camera capture and gallery selection in FaceIDScreen.

Phase 3 — Integration, Testing & Finalization

Phase 3 brought all three teams together for end-to-end integration, system testing, and final modifications.

Full System Integration

- Migrated the AI server from a Flask prototype to the final FastAPI/Uvicorn implementation to support true async WebSocket handling alongside HTTP.
- Validated the complete data path: Flutter HTTP request → AI server mode update → ESP-CAM WebSocket frame → inference → TTS → audio WebSocket → ESP32 speaker.
- Fixed errors and bugs that appeared during integration.
- Resolved integration issues arising from API contract changes during Flask → FastAPI migration.

IoT Team

- Building on the planning conducted in Phase 2, the full integration of hardware components and software logic was executed across all assigned modules.
- Development tasks were allocated among team members, encompassing audio system setup via DFPlayer Mini, real-time self-balancing implementation using the MPU sensor, OLED display configuration, and the structural design of the robot body through 3D modeling in collaboration with the Mechanical team.

Flutter — Integration & Finalization

- Integrated with the finalized FastAPI server — updated endpoint paths and headers accordingly.
- Validated the full face registration flow: photo selection → name input → HTTP POST → AI embedding extraction → Snackbar confirmation → pop.
- Validated real-time movement: WebSocket connect on screen open → D-pad press sends command → D-pad release sends 'S' stop.
- Validated mode activation: Detection and Greeting modes toggle correctly with live server state reflected in CustomModesButtons.
- Added loading overlay (CircularProgressIndicator) and 'Done' button double-submission guard in FaceIDScreen.
- Polished error handling: Snackbar error messages from CameraErrorState and MovementError are surfaced to the user.

8. Implementation Details

8.1 Face Recognition Pipeline

The face recognition pipeline is implemented across `facenet_flow.py` and `FaceId.py`. When the ESP-CAM delivers a video frame, the pipeline executes the following steps:

- The raw frame is converted from BGR to RGB colour space before any processing.
- MediaPipe FaceMesh processes the frame and returns 468 three-dimensional facial landmarks. The bounding box is computed from the min/max x and y coordinates of all landmarks and expanded by 20% in each direction to ensure the full face context is captured.
- The centres of the left and right eyes are estimated by averaging six landmark positions per eye. The roll angle is calculated using atan2 on the vector between the two eye centres. OpenCV's `warpAffine` rotates the full frame to bring the eyes to a horizontal baseline.
- The aligned face region is cropped using the padded bounding box and resized to 160×160 pixels — the input resolution required by InceptionResnetV1.
- Pixel values are normalized from the [0, 255] range to [-1, 1] by the transformation $(x / 127.5) - 1.0$ and arranged into a (1, 3, 160, 160) PyTorch tensor.
- InceptionResnetV1 (VGGFace2 weights) performs a forward pass to produce a 512-dimensional L2-normalized embedding vector.

8.2 Face Database and Search

The face database (`save_loop.py`) persists embeddings as a JSON array in `faces.json`, where each entry contains a name string and a 512-element float array. On startup, the database is loaded into memory as a list of NumPy arrays. The `search_face` function iterates over all stored embeddings and applies the `compare_embeddings` function from `Compare.py`. The default comparison method is L2 (Euclidean) distance with a threshold of 0.8 — two embeddings are classified as the same person if their L2 distance is below this value. Cosine similarity is available as an alternative method.

When a new face is registered via the R mode, `save_face` writes the new entry to `faces.json` and places a reload signal on a multiprocessing Queue. The `face_worker` process monitors this queue and calls `reload_data()` to refresh its in-memory copy of the database without restarting.

8.3 Handshake Detection

`HandShake.py` uses MediaPipe's Hands solution to detect and classify hand gestures. For each detected hand, the detector checks three conditions to classify a high-five or open-palm gesture: (1) all four non-thumb fingers (index, middle, ring, pinky) must have their fingertips above their MCP joints in image-space y-coordinates, indicating the fingers are extended; (2) the Euclidean distance between the index fingertip and pinky fingertip must exceed 0.08 in normalized coordinates, ensuring the fingers are spread; and (3) the hand must be oriented palm-forward, verified by checking the relative x-positions of the index MCP and pinky MCP joints relative to handedness (left vs. right hand). If all three conditions are satisfied, the function returns True.

8.4 Object Detection

Object detection is implemented using Ultralytics YOLOv8 fine-tuned on a custom dataset collected and annotated specifically for the robot's expected deployment environment. The model runs inference on each incoming JPEG frame dispatched by the ESP-CAM WebSocket handler when the system is in Object Detection mode (mode 'O'). Detected object class names are passed to the text-to-speech module for audible announcement.

8.5 Parallel Processing Architecture

The MainFlow module implements a non-blocking parallel processing architecture to ensure that face recognition and hand detection never introduce latency into the video capture loop. Two separate Python processes — `face_worker` and `hand_worker` — are spawned at server startup using the `spawn` start method for CUDA compatibility. Each worker receives frames through a `maxsize=1` Queue (effectively a single-slot buffer) and writes results to a corresponding output Queue, also of `maxsize=1`. The main loop uses non-blocking `put_nowait` and `get_nowait` calls; if a queue is full, the frame is dropped rather than causing a backlog. This design ensures that the ESP-CAM WebSocket handler is never blocked by inference, maintaining responsive video streaming even under computational load.

8.6 Text-to-Speech and Audio Delivery

The `tts.py` module converts text output from the AI pipeline into raw 16-bit PCM audio transmitted over WebSocket to the robot's speaker hardware. It exposes a single function, `text_to_pcm(text, model)`, which returns a `(pcm_bytes, sample_rate)` tuple ready for direct consumption by the ESP32 audio module.

Supported Backends

The module supports five interchangeable backends selectable at call time. All Coqui TTS model instances are lazy-loaded on first use and cached in memory to avoid re-initialization overhead on subsequent calls.

`pyttsx3` is a fully offline engine using the OS native synthesizer. On Windows, the module automatically selects the Microsoft Zira female voice if available. Audio is written to a temporary WAV file, then read back and converted to PCM. This backend offers the lowest latency with no external dependencies.

XTTS v2 is the default production backend. It is a neural voice-cloning model that accepts a reference speaker WAV file and replicates its tonal characteristics, giving BMO a consistent, natural-sounding voice.

FastPitch, Glow-TTS, and Tacotron2-DDC are available as alternative neural backends offering varying trade-offs between voice quality and inference speed for different deployment environments.

PCM Conversion

All backends share a common output path: floating-point audio samples are clipped to `[-1, 1]`, scaled to the 16-bit signed integer range, and serialized to bytes. WAV-

based backends use the soundfile library for reading; Coqui backends convert their array output inline. All Coqui backends operate at 22,050 Hz.

8.7 HTTP / WebSocket Integration Protocol

The server.py FastAPI application exposes two WebSocket routes and one HTTP route. The ESP-CAM WebSocket (/ws/esp_cam) receives binary JPEG frames, decodes them with OpenCV, routes them to the active mode's processing logic, and pushes the TTS audio result to the audio WebSocket. The Flutter HTTP endpoint (/flutter) reads the mode header from the request — F or O switch the global mode state immediately; R triggers the registration workflow, decoding the base64-encoded image from the JSON body, extracting the embedding, saving it to the database, and returning a confirmation with optional TTS confirmation. A GET /status endpoint exposes the current mode for the IoT team's display logic.

8.8 Flutter — Screens

8.8.1 Start Screen

The app's landing screen. Displays the robot's happy 3D illustration over a custom oval-radial gradient background. A single pill button navigates the user into the main application shell. Background uses RadialGradient with OvalGradientTransform(scaleX: 0.6, scaleY: 1.0) producing a horizontally compressed oval glow in #86B4E5.

8.8.2 Home Screen

The main dashboard on the 'Robot' tab. Shows robot status at a glance and exposes the primary interaction button. Status cards (Battery and Power) are rendered as CustomGlassBox widgets. The 'Start the party' button calls MovementCubit.startDanceParty() → HTTP GET to /dance endpoint → SnackBar on result.

8.8.3 Information Screen

A static reference screen showing the project team structure as a 2-column grid of CustomGlassBox cards, with a decorative robot_learn.png illustration pinned to the bottom-right corner.

8.8.4 Face ID Screen

Handles new-user registration. The user selects or captures a photo and enters their first name. Image selection is provided via ImagePicker (camera and gallery). On submission, CameraCubit.registerUser(name, imageFile) encodes the image to base64 and sends an HTTP POST to /flutter with mode header 'R'. A loading state

replaces the button label with '...' to prevent double-submission. Result is surfaced as a `SnackBar` success or error.

8.8.5 Camera Modes Screen

Allows the user to select which AI mode the robot camera should run. Manages active mode in local widget state and dispatches the corresponding `CameraCubit` call.

Button	Effect on Robot
Face ID	Navigates to <code>FaceIDScreen</code> for registration (navigation only)
Detection	Activates object-detection mode on AI server (POST mode: 'O')
Greeting	Activates face-recognition + greeting pipeline (<code>activateGreeting()</code>)

8.8.6 Movement Screen

The movement control tab. Contains the `SwitchCustom` widget which opens a `WebSocket` connection to the IoT ESP32 and sends single-character ASCII commands. Button interaction model: `onTapDown` sends the movement command; `onTapUp` and `onTapCancel` both send 'S' (stop), guaranteeing the robot stops the instant the user's finger leaves the button.

Button / Event	Command Sent
FWD	F — Move Forward
BWD	B — Move Backward
Left	L — Turn Left
Right	R — Turn Right
Release (any button)	S — Stop (auto-sent on finger lift)

8.8.7 Feedback Screen

A post-interaction feedback screen allowing users to rate the robot's performance across three options: 'Perfect' (star icon), 'Good' (thumbs-up icon), and 'Need Enhancement'. Submission handlers are scaffolded and ready to be wired to a feedback endpoint in a future release.

8.9 Flutter — State Management

8.9.1 MovementCubit

Manages high-level robot movement commands triggered from the Home screen. Low-frequency commands (dance, stop) are sent via HTTP GET to named REST endpoints defined in `ApiConstants`.

Method	Description
startDanceParty()	HTTP GET /dance → Loading → Success('The robot is dancing now!') Error
stopMovement()	HTTP GET /move/stop → Initial Error

8.9.2 CameraCubit

Handles all camera-related API calls: mode activation and face registration. Provided via BlocProvider at the widget tree level above CameraModesScreen.

Method	Description
activateMode(String mode)	POST to /flutter with mode header 'F' (face) or 'O' (object)
activateGreeting()	Activates the face-recognition + greeting pipeline on the AI server
registerUser(name, image)	Encodes image to base64, POST to /flutter with mode 'R', name and image in JSON body
resetState()	Resets Cubit to CameraInitial before navigating to FaceIDScreen to clear stale state

8.10 Flutter — Networking

8.10.1 ApiConstants

Single source of truth for all network addresses. Changing the server IP requires editing only one constant.

```
static const String aiServerIp = "192.168.137.1";
static const String aiBaseUrl = "http://192.168.137.1:8000/flutter";
static const String robotIp = "http://192.168.137.1.100";
static const String socketUrl = "ws://192.168.137.1.100:81";
```

8.10.2 ApiService (HTTP)

Wraps the HTTP package for all request/response interactions.

sendAiRequest({mode, body?}) sends HTTP POST to aiBaseUrl with Content-Type: application/json and mode header. sendRobotCommand(endpoint) sends HTTP GET to a named endpoint. The http.Client instance is created once per ApiService instance and reused across calls.

8.10.3 SocketService (WebSocket)

Maintains a persistent WebSocket connection for real-time movement control, scoped to the SwitchCustom widget lifecycle. connect() creates a WebSocketChannel; sendCommand() writes a single ASCII character to channel.sink; disconnect() closes the channel and resets the connection flag.

8.10.4 Communication Summary

Service	Protocol	Used For
ApiService	HTTP POST	Mode activation (F/O/R), face registration
ApiService	HTTP GET	Named movement endpoints (forward / backward / left / right / stop / dance)
SocketService	WebSocket	Real-time directional commands F/B/L/R/S at button-press granularity

8.11 Flutter — Shared Widget Library

Widget	Description
CustomButton	Pill-shaped (borderRadius: 50) pressable container with InkWell ripple. isActive dims to 45% opacity for mode toggle states. Supports optional icon asset and rectangular variant (borderRadius: 20).
CustomGlassBox	Frosted-glass card with BackdropFilter blur (sigmaX:20, sigmaY:10), semi-transparent white fill, and 1.5 px white border. Used as status cards, team cards, and D-pad buttons. Fully parameterized.
CustomTextField	Validated pill-container text input with gradient outer border. Error state shifts border to red and shows inline error label via Stack + Positioned.
CustomModesButtons	372×100 px pill-shaped status display. Dark gradient outer ring + white inner container. Shows current active mode name or latest server response.
CustomNavigationBottom	Main navigation shell using google_nav_bar. Glassmorphism gradient background; active tab shows gradient pill highlight.
OvalGradientTransform	Custom GradientTransform applying non-uniform Matrix4 scale to a RadialGradient, creating an oval glow. Used on the Start screen background.

8.12 Flutter — Key Packages

Package	Purpose
flutter_bloc	BLoC / Cubit state management — Cubits, BlocProvider, BlocConsumer, BlocListener
http	HTTP client for AI server and REST robot endpoints
web_socket_channel	WebSocket client for real-time movement commands to ESP32
image_picker	Camera capture and gallery selection for face registration
google_nav_bar	Animated pill-style bottom navigation bar

Package	Purpose
gap	Concise Gap(n) spacing widget as an alternative to SizedBox(height: n)
vector_math	Matrix4 / Vector3 operations for OvalGradientTransform

9. Conclusion

BMO demonstrates that a compact, affordable robotic platform can deliver meaningful and responsive human-robot interaction when hardware constraints are addressed through thoughtful software architecture. By distributing intelligence across a specialized AI server, an IoT hardware layer, and a mobile application, the project successfully achieved its primary goal of creating a smart, interactive mini-robot that integrates self-balancing, visual perception, and adaptive learning into a single, functional platform. Through the collaborative efforts of the AI, IoT, and Flutter teams, a system has been developed that is not only technically robust but also demonstrates key principles of modern human-robot interaction.

The modular architecture, with a centralized server handling AI processing, has proven effective, enabling real-time responses and decoupling the robot's hardware from its intelligence. The implementation of object detection and facial recognition provides BMO with a high degree of environmental and social awareness. Furthermore, the user registration feature directly addresses the project's goal of adaptability, allowing the robot to personalize its interactions over time.

12. Future Work

- **LLM Conversational Mode (Mode L):** The chatbot module (chatbot.py) is architecturally complete and will be activated as Mode L in the next release. Future work includes integrating speech-to-text input so the robot can engage in fully voice-driven conversation, and evaluating on-device LLM options to reduce API dependency.
- **Feedback-Driven Improvement:** Collecting structured user feedback after each interaction session will enable continuous improvement of recognition thresholds, detection confidence, and conversational response quality.
- **Joystick Control:** Complete `_buildJoystickView()` in `SwitchCustom` — an alternative joystick control layout for the `Movement` screen — for a more intuitive steering experience.
- **Animated Mode Toggle:** Complete `_buildAnimatedToggle()` — an animated visual toggle between manual D-pad and automatic movement modes.
- **Live Video Stream:** Add a live video stream view in the `Camera` screen to show the robot's camera feed in real time within the Flutter app.

- **Local Face Registry:** Add local storage for registered face names so the app can display a list of known individuals without a server query.
- **Connection Status Indicators:** Implement connection-status indicators so the user can see whether the WebSocket and HTTP server are reachable before sending commands.